WHAT IS AN ARRAY?

An array is a collection of elements of the same data type stored in contiguous (continuous) memory locations. Each element is identified by an index, which starts from 0 in Java.

An array allows you to store multiple values in a single variable instead of creating many separate variables.

TYPES OF ARRAYS IN JAVA

There are three main types of arrays:

1. ONE-DIMENSIONAL (1D) ARRAY

A 1D array stores elements in a single row.

Syntax
```
dataType[] arrayName = new dataType[size];
```

2. TWO-DIMENSIONAL (2D) ARRAY

A 2D array stores elements in rows and columns (matrix form).

Syntax
```
dataType[][] arrayName = new dataType[rows][columns];
```

3. JAGGED ARRAY (RAGGED ARRAY)

A Jagged Array is a 2D array in which each row can have a different number of columns.

Syntax
```
dataType[][] arrayName = new dataType[rows][];
```

1. ADDITION
```
public class AddEven {
    public static void main(String[] args) {

        int[] a = {10, 21, 30, 15, 25};
        int[] b = {5, 11, 20, 25, 15};

        System.out.println("Even Numbers After Addition:");

        for (int i = 0; i < a.length; i++) {
            int sum = a[i] + b[i];

            if (sum % 2 == 0) {
                System.out.print(sum + " ");
            }
        }
    }
}
```
```
Even Numbers After Addition:
32 50 40
```

2. SUBTRACTION

```
public class SubEven {
    public static void main(String[] args) {

        int[] a = {20, 35, 40, 50, 65};
        int[] b = {10, 15, 15, 20, 25};

        System.out.println("Even Numbers After Subtraction:");

        for (int i = 0; i < a.length; i++) {
            int sub = a[i] - b[i];

            if (sub % 2 == 0) {
                System.out.print(sub + " ");
            }
        }
    }
}
```

```
Even Numbers After Subtraction:
10 30 40
```

3. MULTIPLICATION

```
public class MulEven {
    public static void main(String[] args) {

        int[] a = {2, 3, 4, 5, 6};
        int[] b = {5, 6, 7, 8, 9};

        System.out.println("Even Numbers After Multiplication:");

        for (int i = 0; i < a.length; i++) {
            int mul = a[i] * b[i];

            if (mul % 2 == 0) {
                System.out.print(mul + " ");
            }
        }
    }
}
```

```
Even Numbers After Multiplication:
18 28 40 54
```

TRAVERSAL OF A STRING IN JAVA

String traversal means visiting each character of a string one by one using a loop.

Syntax

```
for (int i = 0; i < str.length(); i++) {
    char ch = str.charAt(i);
}
```

DEFINITION
String traversal is the process of accessing each character of a string one by one using a loop,
usually with the charAt() method.

PROGRAM TO TRAVERSE A STRING

```
public class StringTraversal {
    public static void main(String[] args) {

        String str = "Java";

        System.out.println("Characters in the string:");

        for (int i = 0; i < str.length(); i++) {
            System.out.println(str.charAt(i));
        }
    }
}
```

OUTPUT

```
Characters in the string:
J
a
v
a
```

. ARRAYLIST

Definition: ArrayList is a dynamic array that can grow or shrink in size.

```
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {

        ArrayList<String> list = new ArrayList<>();

        list.add("Apple");
        list.add("Banana");
        list.add("Mango");

        System.out.println("ArrayList Elements:");
        for (String item : list) {
            System.out.println(item);
        }
    }
}
```

Output

```
ArrayList Elements:
Apple
Banana
Mango
```

2. LINKEDLIST

Definition: LinkedList stores elements as nodes connected by links.

```
import java.util.LinkedList;

public class LinkedListExample {
    public static void main(String[] args) {

        LinkedList<String> list = new LinkedList<>();

        list.add("Red");
        list.add("Green");
        list.add("Blue");

        System.out.println("LinkedList Elements:");
        for (String color : list) {
            System.out.println(color);
        }
    }
}
```

Output

```
LinkedList Elements:
Red
Green
Blue
```

3. VECTOR

Definition: Vector is a dynamic array similar to ArrayList, but it is synchronized.

```
import java.util.Vector;

public class VectorExample {
    public static void main(String[] args) {

        Vector<Integer> vector = new Vector<>();

        vector.add(10);
        vector.add(20);
        vector.add(30);

        System.out.println("Vector Elements:");
        for (int num : vector) {
            System.out.println(num);
        }
    }
}
```

Output

```
Vector Elements:
10
20
30
```

4. STACK

Definition: Stack follows the LIFO (Last In, First Out) principle.

```
import java.util.Stack;
```

```
public class StackExample {
    public static void main(String[] args) {

        Stack<Integer> stack = new Stack<>();

        stack.push(100);
        stack.push(200);
        stack.push(300);

        System.out.println("Stack Elements:");
        System.out.println(stack);

        System.out.println("Top Element: " + stack.peek());

        System.out.println("Removed Element: " + stack.pop());

        System.out.println("Stack After Pop:");
        System.out.println(stack);
    }
}
```

Output

```
Stack Elements:
[100, 200, 300]
Top Element: 300
Removed Element: 300
Stack After Pop:
[100, 200]
```

1. QUEUE (USING LINKEDLIST)

Definition: A Queue follows the FIFO (First In, First Out) principle.

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueExample {
    public static void main(String[] args) {

        Queue<Integer> queue = new LinkedList<>();

        queue.add(10);
        queue.add(20);
        queue.add(30);

        System.out.println("Queue: " + queue);

        System.out.println("Front Element: " + queue.peek());

        System.out.println("Removed Element: " + queue.poll());

        System.out.println("Queue After Removal: " + queue);
    }
}
```

```
Output
Queue: [10, 20, 30]
Front Element: 10
Removed Element: 10
Queue After Removal: [20, 30]
2. PRIORITYQUEUE

Definition: A PriorityQueue stores elements according to their priority
(smallest element first by default).

import java.util.PriorityQueue;

public class PriorityQueueExample {
    public static void main(String[] args) {

        PriorityQueue<Integer> pq = new PriorityQueue<>();

        pq.add(40);
        pq.add(10);
        pq.add(30);
        pq.add(20);

        System.out.println("Priority Queue: " + pq);

        System.out.println("Top Element: " + pq.peek());

        System.out.println("Removed Element: " + pq.poll());

        System.out.println("Priority Queue After Removal: " + pq);
    }
}
Output
Priority Queue: [10, 20, 30, 40]
Top Element: 10
Removed Element: 10
Priority Queue After Removal: [20, 40, 30]
3. DEQUE (USING LINKEDLIST)

Definition: A Deque (Double-Ended Queue) allows insertion and deletion
from both the front and the rear.

import java.util.Deque;
import java.util.LinkedList;

public class DequeExample {
    public static void main(String[] args) {

        Deque<Integer> deque = new LinkedList<>();

        deque.addFirst(10);
        deque.addLast(20);
        deque.addFirst(5);
        deque.addLast(30);

        System.out.println("Deque: " + deque);

        deque.removeFirst();
        deque.removeLast();
```

```
        System.out.println("Deque After Removal: " + deque);
    }
}
Output
Deque: [5, 10, 20, 30]
Deque After Removal: [10, 20]
4. ARRAYDEQUE

Definition: An ArrayDeque is a resizable array implementation of the
Deque interface. It allows insertion and deletion at both ends.

import java.util.ArrayDeque;

public class ArrayDequeExample {
    public static void main(String[] args) {

        ArrayDeque<Integer> deque = new ArrayDeque<>();

        deque.addFirst(100);
        deque.addLast(200);
        deque.addFirst(50);
        deque.addLast(300);

        System.out.println("ArrayDeque: " + deque);

        System.out.println("First Element: " + deque.peekFirst());
        System.out.println("Last Element: " + deque.peekLast());

        deque.removeFirst();
        deque.removeLast();

        System.out.println("ArrayDeque After Removal: " + deque);
    }
}
Output
ArrayDeque: [50, 100, 200, 300]
First Element: 50
Last Element: 300
ArrayDeque After Removal: [100, 200]


1. HASHSET

Definition: HashSet stores unique elements and does not maintain
insertion order.

import java.util.HashSet;

public class HashSetExample {
    public static void main(String[] args) {

        HashSet<String> set = new HashSet<>();

        set.add("Apple");
        set.add("Banana");
        set.add("Mango");
        set.add("Apple");   // Duplicate
```

```
        System.out.println("HashSet Elements:");
        for (String fruit : set) {
            System.out.println(fruit);
        }
    }
}
Output
HashSet Elements:
Apple
Banana
Mango
```

Note: The output order may vary because HashSet does not maintain insertion order.

2. LINKEDHASHSET

Definition: LinkedHashSet stores unique elements and maintains insertion order.

```
import java.util.LinkedHashSet;

public class LinkedHashSetExample {
    public static void main(String[] args) {

        LinkedHashSet<String> set = new LinkedHashSet<>();

        set.add("Apple");
        set.add("Banana");
        set.add("Mango");
        set.add("Apple");   // Duplicate

        System.out.println("LinkedHashSet Elements:");
        for (String fruit : set) {
            System.out.println(fruit);
        }
    }
}
Output
LinkedHashSet Elements:
Apple
Banana
Mango
```

3. TREESET

Definition: TreeSet stores unique elements in ascending (sorted) order.

```
import java.util.TreeSet;

public class TreeSetExample {
    public static void main(String[] args) {

        TreeSet<Integer> set = new TreeSet<>();

        set.add(40);
        set.add(10);
        set.add(30);
```

```
        set.add(20);
        set.add(10);   // Duplicate

        System.out.println("TreeSet Elements:");
        for (int num : set) {
            System.out.println(num);
        }
    }
}
Output
TreeSet Elements:
10
20
30
40
```

1. ENUMERATION

Definition: Enumeration is used to traverse elements of legacy collections like Vector and Stack.

```
import java.util.Vector;
import java.util.Enumeration;

public class EnumerationExample {
    public static void main(String[] args) {

        Vector<String> v = new Vector<>();

        v.add("Java");
        v.add("Python");
        v.add("C++");

        Enumeration<String> e = v.elements();

        System.out.println("Vector Elements:");

        while (e.hasMoreElements()) {
            System.out.println(e.nextElement());
        }
    }
}
Output
Vector Elements:
Java
Python
C++
```

2. ITERATOR

Definition: Iterator is used to traverse any collection in the forward direction.

```
import java.util.ArrayList;
import java.util.Iterator;

public class IteratorExample {
    public static void main(String[] args) {
```

```
        ArrayList<String> list = new ArrayList<>();

        list.add("Apple");
        list.add("Banana");
        list.add("Mango");

        Iterator<String> itr = list.iterator();

        System.out.println("ArrayList Elements:");

        while (itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}
```

Output

```
ArrayList Elements:
Apple
Banana
Mango
```

3. LISTITERATOR

Definition: ListIterator is used to traverse a List in both forward and backward directions.

```
import java.util.ArrayList;
import java.util.ListIterator;

public class ListIteratorExample {
    public static void main(String[] args) {

        ArrayList<String> list = new ArrayList<>();

        list.add("Red");
        list.add("Green");
        list.add("Blue");

        ListIterator<String> litr = list.listIterator();

        System.out.println("Forward Traversal:");

        while (litr.hasNext()) {
            System.out.println(litr.next());
        }

        System.out.println("Backward Traversal:");

        while (litr.hasPrevious()) {
            System.out.println(litr.previous());
        }
    }
}
```

Output

```
Forward Traversal:
Red
Green
Blue
```

```
Backward Traversal:
Blue
Green
Red
```

WHAT IS AN EXCEPTION?

An exception is an unexpected event or error that occurs during the execution of a program and interrupts the normal flow of the program.

Definition

An exception is a runtime event that disrupts the normal execution of a program.

Examples of Exceptions
Dividing a number by zero.
Accessing an invalid array index.
Opening a file that does not exist.
Entering invalid input.
Example of an Exception

```
public class ExceptionExample {
    public static void main(String[] args) {

        int a = 10;
        int b = 0;

        int c = a / b;   // Exception occurs

        System.out.println(c);
    }
}
```

Output

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

HOW TO HANDLE AN EXCEPTION
Exceptions are handled using the following keywords:

try
catch
finally
throw
throws

The most common way is try-catch.

Syntax

```
try {
    // Code that may cause an exception
}
catch(ExceptionType e) {
    // Code to handle the exception
}
```

Example: try-catch

```
public class TryCatchExample {
    public static void main(String[] args) {
```

```
        try {
            int a = 10;
            int b = 0;

            int c = a / b;

            System.out.println(c);
        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        }

        System.out.println("Program continues...");
    }
}
```

Output

```
Cannot divide by zero.
Program continues...
```

finally Block

The finally block executes whether an exception occurs or not.

```
public class FinallyExample {
    public static void main(String[] args) {

        try {
            int a = 10 / 0;
        }
        catch (ArithmeticException e) {
            System.out.println("Exception Handled");
        }
        finally {
            System.out.println("Finally block executed");
        }
    }
}
```

Output

```
Exception Handled
Finally block executed
```

Types of Exceptions

There are two main types of exceptions in Java.

1. CHECKED EXCEPTION

Checked at compile time.

Must be handled using try-catch or throws.

Examples

IOException

FileNotFoundException

SQLException

Example:

```
import java.io.FileReader;
```

```
public class CheckedExceptionExample {
    public static void main(String[] args) {

        try {
            FileReader file = new FileReader("test.txt");
        }
        catch (Exception e) {
            System.out.println("File not found.");
        }
    }
}
```

2. UNCHECKED EXCEPTION

Occurs at runtime.
Not checked by the compiler.

Examples

ArithmeticException
NullPointerException
ArrayIndexOutOfBoundsException
NumberFormatException

Example:

```
public class UncheckedExceptionExample {
    public static void main(String[] args) {

        int[] arr = {10,20,30};

        try {
            System.out.println(arr[5]);
        }
        catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid Array Index");
        }
    }
}
```

Output
Invalid Array Index

JAVA EXCEPTION – TWO-LINE DEFINITIONS

1. EXCEPTION

Exception is an unexpected event that occurs during program execution and interrupts the normal flow of the program.
It can be handled to prevent the program from terminating abnormally.

2. EXCEPTION HANDLING

Exception handling is the process of detecting and handling runtime errors in a program.
It ensures the program continues executing normally after an exception.

3. THROWABLE

Throwable is the root class of all exceptions and errors in Java.
Only objects of the Throwable class or its subclasses can be thrown.

4. ERROR

Error represents serious problems generated by the JVM.
These errors are generally not handled by the programmer.

5. EXCEPTION CLASS

Exception is the parent class of all exceptions in Java.
It represents conditions that an application can catch and handle.

6. CHECKED EXCEPTION

A checked exception is checked by the compiler at compile time.
It must be handled using try-catch or declared with throws.

7. UNCHECKED EXCEPTION

An unchecked exception occurs during program execution (runtime).
It is not checked by the compiler and handling is optional.

8. RUNTIMEEXCEPTION

RuntimeException is the parent class of all unchecked exceptions.
These exceptions occur due to programming mistakes during execution.

9. TRY

The try block contains code that may generate an exception.
It must be followed by a catch or finally block.

10. CATCH

The catch block is used to handle an exception thrown from the try block.
It prevents abnormal program termination.

11. FINALLY

The finally block contains code that always executes.
It runs whether an exception occurs or not.

12. THROW

The throw keyword is used to explicitly create and throw an exception.
It is used inside a method or block.

13. THROWS

The throws keyword declares that a method may throw one or more exceptions.
It passes the responsibility of handling the exception to the caller.

14. ARITHMETICEXCEPTION

ArithmeticException occurs when an illegal arithmetic operation is performed.
A common example is dividing a number by zero.

15. NULLPOINTEREXCEPTION

NullPointerException occurs when a program tries to use a null object reference.
It is one of the most common runtime exceptions.

16. ARRAYINDEXOUTOFBOUNDSEXCEPTION

This exception occurs when an invalid index is used to access an array.
The index is either negative or greater than the array size.

17. NUMBERFORMATEXCEPTION

NumberFormatException occurs when a string cannot be converted into a numeric value.
It usually happens when parsing invalid input.

18. CLASSCASTEXCEPTION

ClassCastException occurs when an object is cast to an incompatible type.
It happens during invalid type conversion.

19. IOEXCEPTION

IOException occurs when an input or output operation fails.
It is a checked exception and must be handled.

20. FILENOTFOUNDEXCEPTION

FileNotFoundException occurs when the specified file cannot be found.
It is a subclass of IOException.

21. SQLEXCEPTION

SQLException occurs when there is an error while accessing a database.
It is a checked exception commonly used in JDBC.

22. CLASSNOTFOUNDEXCEPTION

ClassNotFoundException occurs when the JVM cannot find the specified class.
It is commonly encountered while loading classes dynamically.

23. INTERRUPTEDEXCEPTION

InterruptedException occurs when a thread is interrupted while waiting or sleeping.
It is a checked exception in multithreaded programming.

24. OUTOFMEMORYERROR

OutOfMemoryError occurs when the JVM cannot allocate enough memory for an object.
It is an Error and is generally not handled by applications.

25. STACKOVERFLOWERROR

StackOverflowError occurs when a method calls itself recursively without a stopping condition.

It causes the call stack to overflow.

These short definitions are ideal for Java viva, lab exams, and university theory questions.

1. IOEXCEPTION

Definition:
IOException occurs when an input or output operation fails.
It is a checked exception and must be handled.

2. FILENOTFOUNDEXCEPTION

Definition:
FileNotFoundException occurs when the specified file cannot be found.
It is a subclass of IOException.

3. SQLEXCEPTION

Definition:
SQLException occurs when there is an error while accessing a database.
It is commonly used in JDBC programs.

4. CLASSNOTFOUNDEXCEPTION

Definition:
ClassNotFoundException occurs when the JVM cannot find the specified class.
It commonly occurs while loading database drivers.

5. INTERRUPTEDEXCEPTION

Definition:
InterruptedException occurs when a thread is interrupted while sleeping or waiting.
It is a checked exception in multithreading.

UNCHECKED EXCEPTIONS (RUNTIME EXCEPTIONS)

1. ARITHMETICEXCEPTION

Definition:
ArithmeticException occurs when an illegal arithmetic operation is performed.
The most common cause is dividing a number by zero.

2. NULLPOINTEREXCEPTION

Definition:
NullPointerException occurs when a null object is accessed.
It happens when methods or variables are used on a null reference.

3. ARRAYINDEXOUTOFBOUNDSEXCEPTION

Definition:
ArrayIndexOutOfBoundsException occurs when an invalid array index is accessed.
The index is outside the array's valid range.

4. NUMBERFORMATEXCEPTION

Definition:
NumberFormatException occurs when a string cannot be converted into a number.
It is thrown during invalid numeric conversion.

5. CLASSCASTEXCEPTION

Definition:
ClassCastException occurs when an object is cast to an incompatible type. It happens during invalid type conversion.

```
create database product;
use product;
create table product(
p_id int primary key,
p_name varchar(40),
cost int,
manufacture_name varchar(40),
manufacture_date date
);

insert into product values(1,'lux',34,'HUL','2017-12-12');
insert into product values(2,'locks',1200,'godrej','2018-01-11');

select * from product
```

```
1     lux   34    HUL   2017-12-12
2     locks 1200  godrej      2018-01-11
```

```
      create database employee;
use employee;
create table employee(
e_id int primary key,
name varchar(40)not null,
salary int(10),
bg varchar(40) default 'o+ve',
age int check(age between 18 and 60),
email varchar(40)unique
);


insert into employee values(4,'ravi',36000,'o+ve',48,'ravi@gmail.com');
insert into employee values(5,'simak',45000,'o+ve',18,'simak@gmail.com');
insert into employee values(6,'abuu',46000,'o+ve',20,'abu@gmail.com');
insert into employee values(7,'trajj',47000,'o+ve',19,'traj@gmail.com');
insert into employee values(8,'pradyu',44000,'o+ve',21,'prad@gmail.com');

select * from employee
```

```
4     ravi  36000 o+ve  48    ravi@gmail.com
5     simak 45000 o+ve  18    simak@gmail.com
6     abuu  46000 o+ve  20    abu@gmail.com
7     trajj 47000 o+ve  19    traj@gmail.com
8     pradyu      44000 o+ve  21    prad@gmail.com
```

```
      use student;
create table state(
state_id int primary key,
state_name varchar(40)
);

insert into state values(1,'karnataka'),(2,'kerala'),(3,'tamilnadu');


create table student(
s_id int primary key,
s_name varchar(40),
age int,
state_id int,
foreign key(state_id)references state(state_id));

insert into student
values(101,'ravi',20,1),(102,'simak',21,2),(103,'abu',22,3);

select s.s_id,s.s_name,s.age,st.state_name from student s join state st
on s.state_id=st.state_id;
```

```
101   ravi  20    karnataka
102   simak 21    kerala
103   abu   22    tamilnadu
```